HAFTA 5 • CUMARTESİ

Üretken Yapay Zeka

Difüzyon Modelleri · LLM Prompt Mühendisliği · Flask & FastAPI Deployment

diffusers · Stable Diffusion · llama-cpp-python · Flask · FastAPI · Docker

1000	7B+	REST	4 Dosya
Difüzyon Adımı	LLM Parametre	API Mimari	Python Uygulaması

09:00 — 17:00 · Teoriden pratiğe · Üretken AI'ı deploy et

Günün Planı — Hafta 5 Cumartesi

09:00–09:30 Hafta 4 Özeti & Giriş	09:30–10:45 Difüzyon Modelleri	10:45–11:00 Kısa Mola	11:00–12:00 Stable Diffusion Hugging Face	12:00–13:00 LLM Prompt Mühendisliği
13:00–14:00 Öğle Arası	14:00–15:30 Flask & FastAPI REST API	15:30–16:30 Docker & Deployment	16:30–17:00 Ödev & Kapanış	

Gün Sonu: Stable Diffusion prompt → görüntü → FastAPI endpoint → Docker container → deploy

Difüzyon Modelleri — Teori & Mimari

DDPM · Forward Process · Reverse Process · U-Net · Noise Scheduler

← İLERİ SÜREÇ (Forward Process) $x_0 \rightarrow x_t$

$q(x_t|x_{t-1}) = N(\sqrt{(1-\beta_t)}\cdot x_{t-1},\ \beta_t I)$

Gerçek görüntüye adım adım Gauss gürültüsü eklenir.
β_t küçük → her adımda az gürültü → T=1000 adım sonunda saf N(0,I).

→□ GERİ SÜREÇ (Reverse Process) $x_t \rightarrow x_0$

$p_\theta(x_{t-1}|x_t) = N(\mu_\theta(x_t,t),\ \Sigma_\theta(x_t,t))$

U-Net gürültüyü tahmin eder: $\varepsilon_\theta(x_t,t) \rightarrow x_{t-1}$
Koşullu (text-guided): CLIP embedding koşulu eklenir.

Eğitim Kaybı — MSE Gürültü Tahmini

$L_simple = E[||\varepsilon - \varepsilon_\theta(\sqrt{\bar{\alpha}_t}\cdot x_0 + \sqrt{(1-\bar{\alpha}_t)}\cdot\varepsilon, t)||^2]$

ε: gerçek gürültü · ε_θ: U-Net tahmini · $\bar{\alpha}_t = \prod_i(1-\beta_i)$

DDPM

Ho et al. 2020
1000 difüzyon adımı
MSE gürültü tahmini
Yavaş ama kaliteli

DDIM

Song et al. 2020
Deterministik sampling
50-100 adım yeterli
10-20× hızlanma ✓

LDM

Latent Diffusion
Pixel yerine latent uzay
VAE encoder/decoder
Stable Diffusion temeli

Stable Diffusion

Rombach et al. 2022
CLIP text embedding
Kamuya açık SOTA ✓

Stable Diffusion = LDM + CLIP + CFG · Latent boyut: 64×64×4 · U-Net: 860M param · VRAM: ~4GB

Stable Diffusion — HuggingFace Diffusers

diffusers · StableDiffusionPipeline · ControlNet · img2img · inpainting

stable_diffusion_pipeline.py

```python
from diffusers import StableDiffusionPipeline, DPMSolverMultistepScheduler
import torch

# ── Model Yükleme (FP16 + CUDA) ──────────────────────────────
model_id = 'stabilityai/stable-diffusion-2-1'
pipe = StableDiffusionPipeline.from_pretrained(
    model_id,
    torch_dtype=torch.float16,     # %50 bellek tasarrufu
    safety_checker=None            # demo için kapalı
)
pipe.scheduler = DPMSolverMultistepScheduler.from_config(
    pipe.scheduler.config        # DPM-Solver++: 25 adımda kaliteli
)
pipe = pipe.to('cuda')           # CPU için: 'cpu'

# ── Görüntü Üretimi ──────────────────────────────────────────
prompt = (
    'a futuristic city at sunset, ultra-detailed, 4K, '
    'digital art, cinematic lighting'
)
negative_prompt = 'blurry, low quality, watermark, nsfw'

with torch.autocast('cuda'):     # Otomatik FP16 ↔ FP32
    goruntu = pipe(
        prompt=prompt,
        num_inference_steps=25,  # DPM++ ile 25 yeterli
        guidance_scale=7.5       # CFG ağırlığı (1–20)
```

CFG (Classifier-Free Guidance)

guidance_scale = 1 → Sadece prompt
guidance_scale = 7 → Dengeli (önerilen)
guidance_scale = 15 → Çok katı, artifakt

Formül: $\varepsilon = \varepsilon_{uncon} + w \cdot (\varepsilon_{con} - \varepsilon_{uncon})$

Gürültü Programı (Scheduler)

DDPM: 1000 adım (yavaş, referans)
DDIM: 50 adım deterministik
DPM-Solver++: 25 adım (en hızlı ✓)
Euler Ancestral: 30 adım, yaratıcı

Bellek Optimizasyonu

FP16: torch.float16 → ½ VRAM
pipe.enable_attention_slicing()
pipe.enable_xformers_memory_efficient_attention()
CPU Offload: model.enable_sequential_cpu_offload()

```
pip install diffusers transformers accelerate xformers  ·  GPU: 4GB+  ·  CPU: yavaş ama çalışır  ·  Model: ~5GB
```

LLM Prompt Mühendisliği

Zero-Shot · Few-Shot · Chain-of-Thought · Rol Verme · System Prompt · Temperature

Zero-Shot

```
Soru: Bu yorumu analiz et: 'Harika
film!'
Yanıt: Pozitif
```

Hiç örnek yok, direkt soru.
Basit görevler için yeterli.

Few-Shot

```
Giriş: 'Harika!' → Pozitif
Giriş: 'Berbat!' → Negatif
Giriş: 'Fena değil' → ?
```

2-5 örnek ver, kalıp öğret.
Sınıflandırma, format görevleri.

Chain-of-Thought

```
'Adım adım düşün:
1. ... 2. ... 3. ...
Sonuç:' formatını kullan
```

Matemat., çıkarım, planlama.
Düşünme zinciri hata azaltır.

Rol Verme

```
System: 'Sen uzman bir Python
geliştiricisisin. Kısa, açıklamalı
kod yaz.'
```

Kişilik/uzmanlık ataması.
Tutarlılık ve kalite artar.

Temperature: 0→ deterministik, 0.7→ dengeli, 1.5→ yaratıcı · Top-P (nucleus): olasılık kütlesi eşiği · Max tokens: çıktı limiti

LLM API Entegrasyonu & Yerel Model

OpenAI API · llama-cpp-python · LangChain · Streaming · RAG Temeli

openai_api.py

```python
from openai import OpenAI
client = OpenAI(api_key='sk-...')   # veya os.environ

# ── Chat Completion ──────────────────────────────────────
response = client.chat.completions.create(
    model='gpt-4o-mini',
    messages=[
        {'role':'system','content':'Sen bir Python uzmanısın.'},
        {'role':'user',  'content':'Quicksort yaz, O(n log n)'}
    ],
    temperature=0.3,   max_tokens=500
)
print(response.choices[0].message.content)
```

llama_cpp.py

```python
from llama_cpp import Llama             # Yerel model, GPU opsiyonel
llm = Llama(
    model_path='./models/llama-3-8b.gguf',  # GGUF format
    n_ctx=4096,      # Context penceresi
    n_gpu_layers=35, # GPU katman sayısı (0=CPU)
    verbose=False
)
cevap = llm('Türkçe bir şiir yaz:', max_tokens=200)
print(cevap['choices'][0]['text'])
```

Sistem Prompt Şablonu

System: 'Sen {uzmanlik} alanında uzman bir {rol}sun.
Daima Türkçe cevap ver. {kural}'
User: '{kullanici_sorusu}'
→ f-string veya Jinja2 şablonu

Streaming Yanıt

stream=True → Kelime kelime üretim
for chunk in response: print(chunk.choices[0].delta.content, end='')
UI deneyimini iyileştirir ✓

RAG Temeli (Retrieval)

1. Belgeni chunk'lara böl (512 token)
2. Embedding üret (text-embedding-ada)
3. Vektör DB'ye kaydet (FAISS, ChromaDB)
4. Soru gelince en benzer chunk'ı bul
5. Prompt'a bağlam olarak ekle

pip install openai langchain llama-cpp-python · GGUF: quantize edilmiş model · Q4_K_M: 4-bit, dengeli kalite

Flask ile Model Deployment — REST API

Route · JSON · Blueprint · Error Handler · CORS · Request Validation

app_flask.py

```python
from flask import Flask, request, jsonify, abort
from flask_cors import CORS
import joblib, numpy as np

app = Flask(__name__)
CORS(app)              # Cross-Origin Resource Sharing

# ── Model Yükle (başlangıçta bir kez) ──────────────────────
model = joblib.load('model.pkl')
scaler = joblib.load('scaler.pkl')

# ── Sağlık Kontrolü ────────────────────────────────────────
@app.route('/health', methods=['GET'])
def health():
    return jsonify({'status': 'ok', 'model': 'v1.0'})

# ── Tahmin Endpoint ────────────────────────────────────────
@app.route('/predict', methods=['POST'])
def predict():
    veri = request.get_json(force=True)
    if 'features' not in veri:
        abort(400, description='features alanı eksik')
    X = np.array(veri['features']).reshape(1, -1)
    X_scaled = scaler.transform(X)
    tahmin   = model.predict(X_scaled)
    return jsonify({
        'tahmin':   int(tahmin[0])
```

Flask Proje Yapısı

```
ml_api/
├── app.py        # Ana Flask uygulaması
├── models/
│   ├── model.pkl
│   └── scaler.pkl
├── requirements.txt
├── Dockerfile
└── tests/
    └── test_api.py
```

test_curl.sh

```bash
# Sağlık kontrolü
curl http://localhost:5000/health

# Tahmin isteği
curl -X POST http://localhost:5000/predict \
  -H 'Content-Type: application/json' \
  -d '{"features": [5.1, 3.5, 1.4, 0.2]}'
```

Yanıt Formatı

{"tahmin": 0, "olasilik": 0.9872, "versiyon": "1.0"}
HTTP 200 OK · Content-Type: application/json

```
pip install flask flask-cors  ·  Production: gunicorn -w 4 app:app  ·  pip install gunicorn
```

FastAPI — Modern & Hızlı REST API

Pydantic · Async · OpenAPI/Swagger · Type Hints · Uvicorn

app_fastapi.py

```python
from fastapi import FastAPI, HTTPException, BackgroundTasks
from pydantic import BaseModel, Field, validator
import joblib, numpy as np, asyncio

app = FastAPI(title='ML Model API', version='1.0',
              docs_url='/docs')   # Swagger UI otomatik ✓

# ── Pydantic Şema (otomatik doğrulama) ──────────────────────
class PredictRequest(BaseModel):
    features: list[float] = Field(..., min_length=1, max_length=50)
    model_version: str    = Field(default='v1')

    @validator('features')
    def no_nan(cls, v):
        if any(np.isnan(x) for x in v): raise ValueError('NaN var')
        return v

class PredictResponse(BaseModel):
    tahmin: int
    olasilik: float
    sinif_adi: str

model  = joblib.load('model.pkl')
scaler = joblib.load('scaler.pkl')
SINIFLAR = ['setosa','versicolor','virginica']

# ── Async Endpoint ──────────────────────────────────────────
@app.post('/predict', response_model=PredictResponse)
```

Flask vs FastAPI — Ne Seçmeli?

Özellik	Flask	FastAPI
Async	Sınırlı	Tam ✓
Doğrulama	Manuel	Pydantic ✓
Swagger	Eklenti	Otomatik ✓
Hız	Orta	Yüksek ✓
Olgunluk	Çok yüksek	Yüksek
Öğrenme	Kolay ✓	Orta

Baslatma

```
# Geliştirme
uvicorn app_fastapi:app --reload --port 8000

# Production
uvicorn app_fastapi:app --host 0.0.0.0 --workers 4
```

swagger.py

```
# /docs → Swagger UI (otomatik)
# /redoc → ReDoc dokümantasyonu
# /openapi.json → OpenAPI şeması
```

```
pip install fastapi uvicorn[standard] pydantic  ·  Swagger: http://localhost:8000/docs  ·  Test: httpx
```

Docker ile Container & Deployment

Dockerfile · Multi-stage Build · docker-compose · Hugging Face Spaces

Dockerfile

```
# ── Çok Aşamalı Build ──────────────────────────────────────
FROM python:3.11-slim AS builder
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

FROM python:3.11-slim AS runtime
WORKDIR /app
COPY --from=builder /usr/local/lib/python3.11 \
                    /usr/local/lib/python3.11
COPY . .
EXPOSE 8000
HEALTHCHECK --interval=30s --timeout=3s \
    CMD curl -f http://localhost:8000/health || exit 1
CMD ['uvicorn','app_fastapi:app',
    '--host','0.0.0.0','--port','8000','--workers','2']
```

docker-compose.yml

```
version: '3.8'
services:
  ml-api:
    build: .
    ports:  ['8000:8000']
    volumes: ['./models:/app/models:ro']   # Salt okunur
    - API_KEY=${API_KEY}   # .env dosyasından
    restart: unless-stopped
```

Hugging Face Spaces

app.py + requirements.txt yükle
Gradio veya FastAPI SDK
Ücretsiz GPU (ZeroGPU)
git push → otomatik deploy ✓

Render / Railway

GitHub repo bağla
Dockerfile otomatik algılanır
Ücretsiz tier mevcut
Custom domain + SSL

AWS / GCP / Azure

```
Docker Komutları
docker build -t ml-api .
docker run -p 8000:8000 ml-api
docker-compose up -d
```

Multi-stage build: geliştirme araçları son imaja dahil edilmez → küçük imaj · Sağlık kontrolü zorunlu · .dockerignore ekle

Uçtan Uca ML Deployment Mimarisi

İstemci → API Gateway → Model Servisi → Model Store → Monitoring

İstemci (Web / Mobil)
React / Flutter
HTTP POST /predict

→

API Gateway (Nginx / Kong)
Rate limiting
Auth / SSL

→

FastAPI (Gunicorn/Uvicorn)
Pydantic val.
Async I/O

→

Model Servisi (Inference)
sklearn / torch
Batch predict

→

Model Store (MLflow/S3)
Versiyonlama
A/B test

→

Monitoring (Prometheus/Grafana)
Latency / p99
Drift algılama

Model Versiyonlama (MLflow)

```
mlflow.log_metric('accuracy', 0.942)
mlflow.sklearn.log_model(model, 'model')
```

Modeli kaydet → Staging → Production
A/B test: %10 traffic yeni model

Güvenlik & Auth

API Key: Authorization: Bearer {token}
Rate Limiting: 100 istek/dakika
Input Validation: Pydantic + sanitize
HTTPS zorunlu → Let's Encrypt (ücretsiz)

Monitoring & Alerting

Latency p99 > 500ms → uyarı
Hata oranı > %1 → uyarı
Veri kayması (concept drift) tespiti
Grafana dashboard + Slack entegrasyon

Production checklist: ✓ HTTPS ✓ Auth ✓ Logging ✓ Health check ✓ Rate limit ✓ Model versiyon ✓ Monitoring

Bugünkü Uygulamalar — 4 Python Dosyası

pip install diffusers transformers flask fastapi uvicorn openai langchain llama-cpp-python

Uygulama 01 — Stable Diffusion Pipeline

`uygulama_01_stable_diffusion.py`

- StableDiffusionPipeline: text→image
- Prompt mühendisliği örnekleri
- guidance_scale & num_inference_steps ablasyonu
- img2img: eskiz → gerçekçi görüntü
- Latent uzay interpolasyon

Uygulama 02 — LLM Prompt Mühendisliği

`uygulama_02_llm_prompt.py`

- Zero-shot / Few-shot / CoT karşılaştırması
- OpenAI API + llama-cpp-python yerel model
- System prompt şablonu tasarımı
- Streaming yanıt implementasyonu
- RAG: FAISS + embedding tabanlı QA

Uygulama 03 — Flask & FastAPI Deployment

`uygulama_03_deployment.py`

- Flask: /predict endpoint + Pydantic benzeri doğrulama
- FastAPI: async + Swagger UI otomatik
- Gunicorn multi-worker production
- CORS, rate limit, auth middleware
- API test: pytest + httpx

Uygulama 04 — Docker & CI/CD Pipeline

`uygulama_04_docker_cicd.py`

- Dockerfile: multi-stage optimized build
- docker-compose: servis + nginx proxy
- Hugging Face Spaces deploy script
- MLflow model versiyonlama entegrasyonu
- Prometheus metrik + sağlık kontrolü

GPU opsiyonel — Stable Diffusion CPU'da çalışır (yavaş) · FastAPI test: uvicorn otomatik yüklü · Docker: ücretsiz Desktop

Hafta 5 Cumartesi — Kazanımlar

- ✓ Difüzyon matematiği: ileri/geri süreç ve DDPM kaybı
- ✓ Prompt mühendisliği: Zero-shot, Few-shot, CoT, Rol
- ✓ Flask REST API: route, JSON, CORS, hata yönetimi
- ✓ Docker: Dockerfile, multi-stage, docker-compose
- ✓ Stable Diffusion: CFG, Scheduler seçimi, img2img
- ✓ LLM API: OpenAI + llama-cpp-python yerel model
- ✓ FastAPI: Pydantic, async, Swagger UI otomatik
- ✓ Production: auth, rate limit, monitoring, MLflow

Hafta 5 Pazar: Kapanış Projeleri Sunumu & Kurs Sertifika Töreni